CSP-J 2024 (Junior)

2024-10-26, 8:30–12:00 (3.5 hours)

Problem 1: Poker Cards (扑克牌)

Time limit: 1000 ms. Memory limit: 512 MB.

Little P borrowed a deck of n poker cards from his classmate Little Q.

In this problem we do not consider the jokers. Then every card has two attributes: a suit and a rank. There are 4 suits: diamonds, clubs, hearts and spades. There are 13 ranks; from smallest to largest they are `A 2 3 4 5 6 7 8 9 T J Q K`. Note: in this problem the rank 10 is written as `T`.

We call a deck of poker cards complete if and only if for every suit and every rank there is exactly one card with that suit and that rank. Hence a complete deck has exactly $4 \times 13 = 52$ cards. The following picture shows all 52 cards of a complete deck.

(The picture, omitted here, displays the 52 cards arranged in four rows, one row per suit, each row running through the 13 ranks.)

The deck Little P borrowed may not be complete, so Little P plans to borrow some more cards from his classmate Little S. You may assume that Little S has infinitely many cards of every kind, so Little P can choose the borrowed cards arbitrarily. Little P wants to know the minimum number of cards he must borrow from Little S so that, from the cards borrowed from Little S and from Little Q, one can select 52 cards that form a complete deck.

To make the input convenient, we use the character `D` for diamonds, `C` for clubs, `H` for hearts and `S` for spades. In this way every card can be represented by a string of length 2, whose first character gives the suit of the card and whose second character gives its rank. For example, `CA` denotes the club `A` and `ST` denotes the spade `T` (the spade 10).

Input

The first line of input contains an integer n, the number of cards.

The next n lines follow:

Each line contains a string of length 2 describing one card, whose first character describes its suit and whose second character describes its rank.

Output

Output one line containing one integer: the minimum number of cards that still have to be borrowed from Little S in order to assemble a complete deck.

Sample 1 input

```
1
SA
```

Sample 1 output

```
51
```

Explanation of sample 1

This deck contains one spade `A`. Little P still has to borrow the 51 cards other than the spade `A` to form a complete deck.

Sample 2 input

```
4
DQ
H3
DQ
DT
```

Sample 2 output

```
49
```

Explanation of sample 2

This deck contains two diamond `Q`, one diamond `T` (the diamond 10) and one heart 3. Little P still has to borrow the 49 cards other than the heart 3, the diamond `T` and the diamond `Q`.

Sample 3

See `poker/poker3.in` and `poker/poker3.ans` in the contestant's directory. (On Luogu these files are provided in the attachment `poker.zip`.)

Explanation of sample 3

This deck is complete, so no further card needs to be borrowed.

This sample satisfies the condition that all cards are given in increasing order of rank, and cards of equal rank are given in the order diamonds, clubs, hearts, spades.

Constraints

For all test data it is guaranteed that $1 \leq n \leq 52$, and that each of the n input strings represents a valid poker card, i.e. the string has length 2, its first character is one of `D C H S`, and its second character is one of `A 2 3 4 5 6 7 8 9 T J Q K`.

Test case number	$n \leq$	Special property
1	1	A
$2 \sim 4$	52	A
$5 \sim 7$	52	B
$8 \sim 10$	52	none

Special property A: it is guaranteed that the n input cards are pairwise distinct.

Special property B: it is guaranteed that all cards are given in increasing order of rank, and cards of equal rank are given in the order diamonds, clubs, hearts, spades.

Problem 2: Map Exploration (地图探险)

Time limit: 1000 ms. Memory limit: 512 MB.

Little A plans to go exploring in a jungle. The geography of the jungle is very complicated, so to avoid getting lost he first sends a robot to scout the way.

The map of the jungle can be represented by a table of characters with n rows and m columns. We denote the coordinates of the position in row i and column j by (i, j) ($1 \le i \le n$, $1 \le j \le m$). If the character at this position is `x`, then there is an obstacle at this position and it cannot be passed. Otherwise, if the character at this position is `.`, then this position is empty ground and can be passed.

The state of the robot consists of two parts: a position and a direction. The position is described by coordinates (x, y) ($1 \le x \le n$, $1 \le y \le m$), meaning that the robot is at the position in row x and column y of the map. The direction is given by an integer d in $0 \sim 3$, where $d = 0$ means facing east, $d = 1$ means facing south, $d = 2$ means facing west and $d = 3$ means facing north.

Initially the robot is at position (x_0, y_0) facing direction d_0. It is guaranteed that the initial position of the robot is empty ground. The robot will then perform k operations. In each step the robot acts according to the following pattern:

1. Suppose the robot is currently at position (x, y) facing direction d. Then the next position (x', y') in its direction is defined as follows: if $d = 0$, let $(x', y') = (x, y + 1)$; if $d = 1$, let $(x', y') = (x + 1, y)$; if $d = 2$, let $(x', y') = (x, y - 1)$; if $d = 3$, let $(x', y') = (x - 1, y)$.

2. Next, the robot checks whether its next position lies inside the map and is empty ground. Concretely, it checks whether (x', y') satisfies $1 \le x' \le n$, $1 \le y' \le m$, and position (x', y') is empty ground. If the condition holds, the robot moves one step forward. Its new position becomes (x', y') and its direction is unchanged. If the condition does not hold, it performs a "turn right" operation. That is, let $d' = (d + 1) \bmod 4$ (i.e. the remainder of $d + 1$ divided by 4); the position stays the same, but the direction changes from d to d'.

Little A wants to know how many positions on the map have been visited by the robot (including the starting position) after the robot has performed the k operations.

Input

This problem has multiple test cases.

The first line of input contains a positive integer T, the number of test cases.

Then T test cases follow, each in the following format:

The first line contains three positive integers n, m, k. Here n, m are the numbers of rows and columns of the map, and k is the number of operations the robot performs.

The second line contains two positive integers x_0, y_0 and a non-negative integer d_0.

The next n lines each contain a string of length m. It is guaranteed that the strings contain only the two characters `x` and `.`. The y-th character of the x-th string corresponds to position (x, y). If this character is `x`, the position is an obstacle; otherwise it is empty ground. The data guarantee that the initial position of the robot is empty ground.

Output

For each test case: output one line containing one positive integer, the number of positions on the map visited by the robot (including the starting position).

Sample 1 input

```
2
1 5 4
1 1 2
....x
5 5 20
1 1 0
.....
.xxx.
.x.x.
..xx.
x....
```

Sample 1 output

```
3
13
```

Explanation of sample 1

This sample contains two test cases. In the first test case the state of the robot changes as follows:

1. Initially the robot is at position $(1, 1)$ facing west (denoted by the number 2).
2. In the first step the robot finds that its next position $(1, 0)$ is not inside the map, so it performs a "turn right" operation. Its position is still $(1, 1)$, but it now faces north (denoted by the number 3).
3. In the second step the robot finds that its next position $(0, 1)$ is not inside the map, so it again performs a "turn right" operation. Its position is still $(1, 1)$, but it now faces east (denoted by the number 0).
4. In the third step the robot finds that its next position $(1, 2)$ is inside the map and is empty ground. Hence it moves one step east. Its position becomes $(1, 2)$ and it still faces east.
5. In the fourth step the robot finds that its next position $(1, 3)$ is inside the map and is empty ground. Hence it moves one step east. Its position becomes $(1, 3)$ and it still faces east.

Therefore after four steps the robot has visited three positions, namely $(1, 1), (1, 2), (1, 3)$.

In the second test case the operations performed by the robot in order are: move east to $(1, 2)$, move east to $(1, 3)$, move east to $(1, 4)$, move east to $(1, 5)$, turn right, move south to $(2, 5)$, move south to $(3, 5)$, move south to $(4, 5)$, move south to $(5, 5)$, turn right, move west to $(5, 4)$, move west to $(5, 3)$, move west to $(5, 2)$, turn right, move north to $(4, 2)$, turn right, turn right, move south to $(5, 2)$, turn right, turn right.

Sample 2

See `explore/explore2.in` and `explore/explore2.ans` in the contestant's directory. (On Luogu these files are provided in the attachment `explore.zip`.)

This sample satisfies the constraints of test cases $3 \sim 4$.

Sample 3

See `explore/explore3.in` and `explore/explore3.ans` in the contestant's directory.
This sample satisfies the constraints of test case 5.

Sample 4

See `explore/explore4.in` and `explore/explore4.ans` in the contestant's directory.
This sample satisfies the constraints of test case 6.

Sample 5

See `explore/explore5.in` and `explore/explore5.ans` in the contestant's directory.
This sample satisfies the constraints of test cases $8 \sim 10$.

Constraints

For all test data it is guaranteed that $1 \leq T \leq 5$, $1 \leq n, m \leq 10^3$, $1 \leq k \leq 10^6$, $1 \leq x_0 \leq n$, $1 \leq y_0 \leq m$, $0 \leq d_0 \leq 3$, and the starting position of the robot is empty ground.

Test case number	n	m	k	Special property
1	$= 1$	≤ 2	$= 1$	none
2	$= 1$	≤ 2	$= 1$	none
3	$\leq 10^2$	$\leq 10^2$	$= 1$	none
4	$\leq 10^2$	$\leq 10^2$	$= 1$	none
5	$= 1$	$\leq 10^3$	$\leq 2 \times 10^3$	all positions on the map are empty ground
6	$= 1$	$\leq 10^3$	$\leq 2 \times 10^3$	none
7	$\leq 10^3$	$\leq 10^3$	$\leq 10^6$	all positions on the map are empty ground
8	$\leq 10^3$	$\leq 10^3$	$\leq 10^6$	none
9	$\leq 10^3$	$\leq 10^3$	$\leq 10^6$	none
10	$\leq 10^3$	$\leq 10^3$	$\leq 10^6$	none

Problem 3: Little Sticks (小木棍)

Time limit: 1000 ms. Memory limit: 512 MB.

Little S likes collecting little sticks. After collecting n little sticks of equal length, having nothing else to do, he used them to form digits. The way each digit is formed from little sticks is shown in the figure below.

(The figure, omitted here, shows the digits 0 to 9 in the usual seven-segment style; the numbers of sticks used by the digits $0, 1, 2, 3, 4, 5, 6, 7, 8, 9$ are $6, 2, 5, 5, 4, 5, 6, 3, 7, 6$ respectively.)

Now Little S wants to form a positive integer satisfying the following conditions:

- forming this number uses exactly n little sticks;
- the number formed has no leading 0;
- subject to the two conditions above, the number is as small as possible.

Little S wants to know what this number is, but n is very large and just sorting out the sticks exhausted Little S, so you need to help him solve this problem. If no positive integer satisfies the conditions above, you need to output -1 to report this.

Input

This problem has multiple test cases.

The first line of input contains a positive integer T, the number of test cases.

Then T test cases follow, each in the following format:

One line containing an integer n, the number of sticks.

Output

For each test case: output one line; if a positive integer satisfying the requirements exists, output this number, otherwise output -1.

Sample 1 input

```
5
1
2
3
6
18
```

Sample 1 output

```
-1
1
7
6
208
```

Explanation of sample 1

- For the first test case, no positive integer can be formed with exactly one little stick, so output -1.
- For the fourth test case, note that 0 is not a valid answer. Forming 9, 41 or 111 each requires exactly 6 little sticks, but they are not the solutions with the smallest number formed.
- For the fifth test case, forming 208 requires $5 + 6 + 7 = 18$ little sticks. It can be shown that forming any positive integer smaller than 208 requires a number of little sticks different from 18. Note that although forming 006 also requires 18 little sticks, this number has a leading zero and is therefore not a valid answer.

Constraints

For all test data it is guaranteed that $1 \leq T \leq 50$, $1 \leq n \leq 10^5$.

Test case number	$n \leq$	Special property
1	20	none
2	50	none
3	10^3	A
$4, 5$	10^5	A
6	10^3	B
$7, 8$	10^5	B
9	10^3	none
10	10^5	none

Special property A: it is guaranteed that n is a multiple of 7 and $n \geq 100$.
Special property B: it is guaranteed that there exists an integer k with $n = 7k+1$, and $n \geq 100$.

Problem 4: Chain (接龙)

Time limit: 2000 ms. Memory limit: 512 MB.

Having grown used to playing idiom chains, Little J and his friends invented a new chain rule.

In total n people take part in this chain game; the i-th person receives an integer sequence S_i as his word bank.

A game consists of several rounds, and each round follows these rules:

- Some person p among the n people plays the round using his word bank S_p. If this is not the first round of the game, the person playing this round must differ from the person of the previous round, but may coincide with the person of the round before the previous one or any earlier round.
- The player chooses a contiguous subsequence A of S_p of length in $[2, k]$ as the chain sequence of this round, where k is a given constant. If this is the first round of the game, A must begin with the element 1; otherwise A must begin with the last element of the chain sequence of the previous round.
  - A sequence A is a contiguous subsequence of a sequence S if and only if A can be obtained by deleting some elements (possibly none) from the beginning and from the end of S.

To emphasise cooperation, Little J gives the n participants q tasks. The j-th task requires the n people to play one game in which exactly r_j rounds are played and the last element of the chain sequence of the last round is exactly c_j. To make sure that the tasks are feasible, Little J asks you to determine whether each of the q tasks can be completed, i.e. whether there exists a possible course of the game satisfying the task's conditions.

Input

This problem has multiple test cases.

The first line of input contains a positive integer T, the number of test cases.

Then T test cases follow, each in the following format:

The first line contains three integers n, k, q: the number of participants, the upper bound on the length of a chain sequence, and the number of tasks.

The next n lines follow:

The i-th line contains (l_i+1) integers $l_i, S_{i,1}, S_{i,2}, \ldots, S_{i,l_i}$, where the first integer l_i is the length of the sequence S_i and the next l_i integers describe the sequence S_i.

The next q lines follow:

The j-th line contains two integers r_j, c_j describing one task.

Output

For each task: output one line containing one integer, 1 if the task can be completed and 0 otherwise.

Sample 1 input

```
1
3 3 7
5 1 2 3 4 1
3 1 2 5
3 5 1 6
1 2
1 4
2 4
3 4
6 6
1 1
7 7
```

Sample 1 output

```
1
0
1
0
1
0
0
```

Explanation of sample 1

In what follows we write $\{A_i\} = \{A_1, A_2, \ldots, A_r\}$ for all the chain sequences of one game, and $\{p_i\} = \{p_1, p_2, \ldots, p_r\}$ for the numbers of the corresponding players. Since all characters are single digits, for convenience we write sequences directly as digit strings.

- For the first query, $p_1 = 1$, $A_1 = 12$ is a valid course of the game satisfying the conditions.
- For the second query, it can be shown that the task cannot be completed. Note that $p_1 = 1$, $A_1 = 1234$ is not a valid course of the game, since then $|A_1| = 4 > k$.
- For the third query, $\{p_i\} = \{2, 1\}$, $\{A_i\} = \{12, 234\}$ is a valid course of the game satisfying the conditions.
- For the fourth query, it can be shown that the task cannot be completed. Note that $\{p_i\} = \{2, 1, 1\}$, $\{A_i\} = \{12, 23, 34\}$ is not a valid course of the game: although all chain sequences have length at most k, the second and third rounds are played by the same person, which is not allowed.
- For the fifth query, $\{p_i\} = \{1, 2, 3, 1, 2, 3\}$, $\{A_i\} = \{12, 25, 51, 12, 25, 516\}$ is a valid course of the game satisfying the conditions.
- For the sixth query, it can be shown that the task cannot be completed. Note that every chain sequence must have length at least 2, so $A_1 = 1$ is not a valid course of the game.
- For the seventh query, the character `7` does not occur in anyone's word bank, so the task obviously cannot be completed.

Sample 2

See `chain/chain2.in` and `chain/chain2.ans` in the contestant's directory. (On Luogu these files are provided in the attachment `chain.zip`.)

This sample satisfies the special property of test case 1.

Sample 3

See `chain/chain3.in` and `chain/chain3.ans` in the contestant's directory.

This sample satisfies the special property of test case 2.

Sample 4

See `chain/chain4.in` and `chain/chain4.ans` in the contestant's directory.

This sample satisfies special property A; moreover its first two test cases satisfy $n \leq 1000$, $r \leq 10$, the sum of the lengths of all word banks within one test case ≤ 2000, and $q \leq 1000$.

Sample 5

See `chain/chain5.in` and `chain/chain5.ans` in the contestant's directory.

This sample satisfies special property B; moreover its first two test cases satisfy $n \leq 1000$, $r \leq 10$, the sum of the lengths of all word banks within one test case ≤ 2000, and $q \leq 1000$.

Sample 6

See `chain/chain6.in` and `chain/chain6.ans` in the contestant's directory.

This sample satisfies special property C; moreover its first two test cases satisfy $n \leq 1000$, $r \leq 10$, the sum of the lengths of all word banks within one test case ≤ 2000, and $q \leq 1000$.

Constraints

For all test data it is guaranteed that:

- $1 \leq T \leq 5$;
- $1 \leq n \leq 10^5$, $2 \leq k \leq 2 \times 10^5$, $1 \leq q \leq 10^5$;
- $1 \leq l_i \leq 2 \times 10^5$, $1 \leq S_{i,j} \leq 2 \times 10^5$;
- $1 \leq r_j \leq 10^2$, $1 \leq c_j \leq 2 \times 10^5$;
- letting $\sum l$ denote the sum of all l_i within one test case, $\sum l \leq 2 \times 10^5$.

Test case	$n \leq$	$r \leq$	$\sum l \leq$	$q \leq$	Special property
1	10^3	1	2000	10^3	none
$2, 3$	10	5	20	10^2	none
$4, 5$	10^3	10	2000	10^3	A
6	10^5	10^2	2×10^5	10^5	A
$7, 8$	10^3	10	2000	10^3	B
$9, 10$	10^5	10^2	2×10^5	10^5	B
$11, 12$	10^3	10	2000	10^3	C
$13, 14$	10^5	10^2	2×10^5	10^5	C
$15 \sim 17$	10^3	10	2000	10^3	none
$18 \sim 20$	10^5	10^2	2×10^5	10^5	none

Special property A: it is guaranteed that $k = 2 \times 10^5$.

Special property B: it is guaranteed that $k \leq 5$.

Special property C: it is guaranteed that within one test case, the total number of occurrences of any one character in all word banks does not exceed 5.

Source

The problems were translated from the official statements as hosted on Luogu (the contest is not on QOJ):

- Problem 1: `https://www.luogu.com.cn/problem/P11227`
- Problem 2: `https://www.luogu.com.cn/problem/P11228`
- Problem 3: `https://www.luogu.com.cn/problem/P11229`
- Problem 4: `https://www.luogu.com.cn/problem/P11230`

Time and memory limits are those shown on Luogu. In the subtask tables, cells that Luogu displays as merged with the cell above ("^") have been filled in with the repeated value. The two figures of the original statements (the 52 cards of Problem 1 and the stick digits of Problem 3) are replaced by textual descriptions.